

CMPM 125 Report

Team Members:

Edison Chan, (Previous Partner was Al Wen, but I didn't know he would later drop the class)

Project & Description (Marble Game)

For this assignment, I did the marble game, for which I created a simple, 3 level course with a limited variety of intractable elements in order to demonstrate the physics functionality.

Player

- Wouldn't be a game without the player. The player is simply a **dynamic rigidbody** attached to a mesh collider. For more marble-like physics, I created a custom **physics material**, with **reduced static friction and bounciness**, so the player felt like a heavier object which rolled/slid around in response to forces.
- Speaking of which, the player's movement is very special. I implemented a **Cinemachine Camera** for better in-game camera control, and by combining the camera's directional vector and omnidirectional input, The player **add force** in or relative to the camera's direction
- Internally, the player uses **OnCollisionEnter()** in order to keep track of collisions with certain objects, like Hazards or Ramps, which is done by checking the tag of the game object it collided with, and acting accordingly.

Places of Interest

- Both Checkpoints, Goalposts, and Coins have **rigidbody triggers**, and use **OnTriggerEnter()** in order to register whether or not they've been touched, and check the triggering gameobject's tag in order to check whether or not it was a player. They then invoke a method in a static variable or Event Bus in order to update the rest of the game

Hazards and Obstacles

- Lava is the main hazard this game has. It's a bright red plane with a **box collider**. The player will check if it collided with a game object tagged as a "Hazard", and if it did, it will call the death function and respawn at the nearest checkpoint
- Ramps are a hazard, obstacle, or helper. Similar to lava, it contains a **box collider**, and the player will internally check if it collided with a game object tagged as a "Ramp". But instead of dying, the player's **linear velocity** will be doubled, launching it in the direction it was going in.

- In order to make ramps feel powerful but also not launch the player into space, I created another **physics material** for the ramps, with **reduced dynamic friction and zero bounciness**.

Conclusion

Due to my unique circumstances, I ended up being the only person on this team, so I did everything. The professor allowed me to either join a new group or keep going, and I chose to continue alone. I've had prior experience with Unity 2D, this project has shown me that Unity 3D is a completely different world. Sure, many of the design patterns I use for internal systems still hold up, but 3D game worlds rely much more heavily on physics and physics-based objects than a 2D world, where physics is pretty much optional.

I used to think the Unity Engine has some of the best, most forgiving physics a game engine could offer, but this project has made me aware of a few of its shortcomings. A small issue was getting used syncing gameobject transform data, rigidbody positional data, and fixedUpdate. I learned that any changes made to the transform position only update to the rigidbody AFTER fixedUpdate. Since Update and fixedUpdate operate at different frequencies, there are tiny moments where these two positions are off. That was a simple fix that should've been obvious, but something significantly more odd to me was how certain mesh colliders warped when their gameobjects became children of other gameobjects. This is why some parts of my game use box colliders instead of mesh colliders; the box colliders for some reason more resistant to this collider warping.